# MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY

Department of Electronics and Communication Engineering




## LAB FILE

## SIGNALS & SYSTEMS

## (ECC – 259)

**Semester 3rd**

| Submitted By | Submitted To |
|---|---|
| Ronit Chauhan | **Dr.** Neeru Rathee |
| 02315002825 | Professor |
| ECE-1 (3rd Semester) | Department Of ECE |

# INDEX

| Exp no. | Date | Exp name | Page no. | Teacher's sign | Remarks |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |

# Experiment : 1

## AIM:- Introduction to MATLAB and its basic commands.

**THEORY:-**

**Introduction to MATLAB:**

MATLAB (Matrix Laboratory) is a high-level programming language and interactive environment developed by MathWorks. It is designed primarily for **numerical computing, data analysis, algorithm development, and matrix manipulations**. MATLAB treats almost every piece of data as a matrix or array by default, removing the need to explicitly declare data types before using variables.

MATLAB is particularly useful in Signal and System analysis because signals can be represented as vectors or matrices and can be easily generated, manipulated, and plotted. Operations such as addition, multiplication, convolution, Fourier transform, Laplace transform, sampling, and filtering can be performed using built-in MATLAB functions.

The main components of the MATLAB environment are:

1. **Command Window:** Used to enter and execute MATLAB commands directly.
2. **Workspace:** Displays the variables currently stored in MATLAB.
3. **Command History:** Stores previously executed commands.
4. **Current Folder:** Shows the files and folders in the current working directory.
5. **Editor:** Used to write and save MATLAB programs, called **scripts** or **M-files**.

**ii. Basic MATLAB Commands:**

| Command | Purpose | Example |
|---|---|---|
| `clc` | Clears the Command Window | `clc` |
| `clear` | Removes variables from Workspace | `clear` |
| `close all` | Closes all open figure windows | `close all` |
| `who` | Displays variables in Workspace | `who` |
| `whos` | Displays detailed information about variables | `whos` |
| `help` | Provides help about a command | `help plot` |
| `disp()` | Displays a value or message | `disp('Hello')` |
| `input()` | Takes input from the user | `x=input('Enter x=')` |
| `length()` | Finds the length of a vector | `length(x)` |
| `size()` | Gives dimensions of a matrix | `size(A)` |
| `plot()` | Plots continuous data | `plot(x,y)` |
| `stem()` | Plots discrete data | `stem(n,x)` |
| `xlabel()` | Labels x-axis | `xlabel('Time')` |
| `ylabel()` | Labels y-axis | `ylabel('Amplitude')` |
| `title()` | Adds a title to a plot | `title('Signal')` |
| `;` | At the end of a command supresses the Display of its result | `a = 10; (stores 10 in a but does not display it)` |

The Command Window is the simplest way to execute MATLAB commands.

**iii. Basic Mathematical Operations:**

MATLAB can perform basic arithmetic operations using:

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /
- Power: ^

For example:

```
a = 10;
b = 5;

sum = a + b
difference = a - b
product = a * b
quotient = a / b
power = a^2
```

MATLAB also supports commonly used mathematical functions such as. sqrt(), sin(), cos(), tan(), log(), exp(), and abs(). For example: sqrt(25) gives 5.

**iv. Vectors and Matrices:**

A row vector can be created using:

```
x = [1 2 3 4 5]
```

A column vector can be created using:

```
x = [1; 2; 3; 4; 5]
```

A matrix can be defined as:

```
A = [1 2 3; 4 5 6; 7 8 9]
```

Here, spaces separate elements in a row and semicolons separate different rows. Individual elements of a matrix can be accessed using their row and column positions. For example, A(2,3) represents the element in the second row and third column. MATLAB provides useful commands such as zeros(), ones(), and eye() for generating special matrices. For example:

zeros(3,3) creates a 3 × 3 matrix of zeros.

ones(2,3) creates a 2 × 3 matrix of ones.

eye(3) creates a 3 × 3 identity matrix.

: **(Colon Operator)** : Generates regularly spaced vector ranges using the format start:increment:end.

- *Example:* x = 1:2:9 generates [1, 3, 5, 7, 9].

The colon operator is very useful in Signal and System experiments for creating the time axis or sample index.

**v. Plotting in MATLAB:**

MATLAB provides powerful graphical facilities for displaying signals and mathematical functions. The plot() command is commonly used for plotting continuous-looking data. For example: x = 0:0.1:10; y = sin(x); plot(x,y); plots a sine wave.

The commands xlabel(), ylabel(), and title() are used to provide labels and a title to the graph. The grid on command adds a grid to the graph.

For discrete-time signals, the stem() command is commonly used:

stem(n,x);

It represents the samples as individual stems and is therefore very useful for discrete signal representation.

**vi. MATLAB in Signal and System Analysis:**

MATLAB provides many built-in functions specifically useful for signal processing. Signals can be represented using vectors and functions. Common operations include signal addition, multiplication, shifting, scaling, convolution, and transformation.

For example, convolution of two sequences can be performed using:

y = conv(x,h);

where x and h are the input sequences and y is the resulting convolution sequence.

MATLAB also provides functions such as fft() for calculating the **Fast Fourier Transform,** ifft() for the **inverse Fourier transform,** and filter() for digital filtering.

Thus, MATLAB significantly reduces the mathematical and computational effort involved in signal and system analysis.

**vii. conclusion**:

MATLAB is a powerful and convenient software environment for numerical computation, programming, visualization, and signal processing. Its matrix-based structure, simple syntax, built-in mathematical functions, and powerful plotting facilities make it highly suitable for studying Signals and Systems. Basic commands such as clc, clear, disp, plot, stem, zeros, ones, and the colon operator form the foundation for performing MATLAB-based laboratory experiments. Learning these commands provides the necessary foundation for implementing and analyzing various continuous-time and discrete-time signals in MATLAB, performing various Signals and Systems experiments using MATLAB.

## A. Generation and Plotting of Sine and Cosine Signals using MATLAB.

## Code:

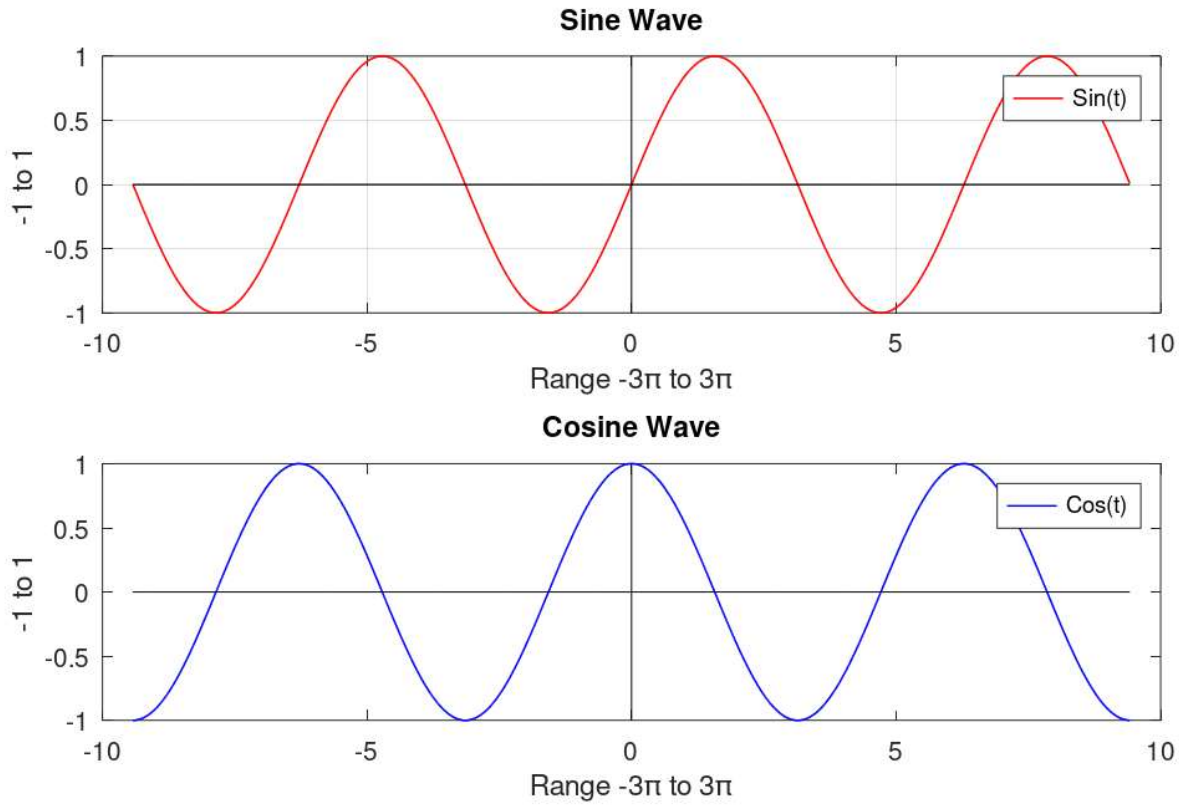
```
clc;
clear all;
close all;

t = [-3*pi:0.01:3*pi];
x = sin(t);
y = cos(t);

subplot(2,1,1)
plot(t,x,'Color','Red');
hold on;
plot(t,0*t,'Color','Black');          % y = 0  -> X-axis
plot([0 0],[-1 1],'Color','Black');   % x = 0  -> Y-axis
hold off;
xlabel('Range -3pi to 3pi')
ylabel('-1 to 1')
title('Sine Wave')

subplot(2,1,2)
plot(t,y,'Color','Blue');
hold on;
plot(t,0*t,'Color','Black');          % y = 0  -> X-axis
plot([0 0],[-1 1],'Color','Black');   % x = 0  -> Y-axis
hold off;
xlabel('Range -3pi to 3pi')
ylabel('-1 to 1')
title('Cosine Wave')
```

## Output:


Sine Wave
Sin(t)
-1 to 1
1
0.5
0
-0.5
-1
-10
-5
0
5
10
Range -3π to 3π
Cosine Wave
Cos(t)
-1 to 1
1
0.5
0
-0.5
-1
-10
-5
0
5
10
Range -3π to 3π

## B. Matrix Addition and Matrix Multiplication using MATLAB.

## Code:

```
clc;
clear all;
close all;
A = [1 2 3; 4 5 6; 7 8 9];
B = [11 12 13; 14 15 16; 17 18 19];

X = A + B;
Y = A * B;

disp('Matrix A is :');
disp(A);
disp('Matrix B is :');
disp(B);

disp('Matrix Addition of A and B is');
disp(X);
disp('Matrix Multiplication of A and B is');
disp(Y);
```

## Output:

```
Matrix A is :
   1   2   3
   4   5   6
   7   8   9
Matrix B is :
   11   12   13
   14   15   16
   17   18   19
Matrix Addition of A and B is
   12   14   16
   18   20   22
   24   26   28
Matrix Multiplication of A and B is
    90    96   102
   216   231   246
   342   366   390
```

## C. Implementation of for loop for generating table of 2 using MATLAB.

## Code:

```
clc;
clear all;
close all;

for i = 1:10
    fprintf('2 x %d = %d\n',i,i*2);
    if i == 10
        disp('2 KA TABLE DONE')
    end
end
```

## Output:

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
2 KA TABLE DONE
```

# Experiment : 2

## AIM:- Plot unit step, unit impulse, unit ramp, exponential and sinusoidal signals.

## a. Generation of unit step, unit impulse and unit ramp signal using MATLAB.

## Code:

```
clc;
clear all;
close all;

t = [-5:1:5];
t_leng = length(t);

% Unit Impulse
for i = 1:t_leng

    if t(i) == 0
        x(i) = 1;
    else
        x(i) = 0;
    end

end

subplot(3, 2, 1)
plot(t, x, 'r', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Unit Impulse Continuous')
subplot(3, 2, 2)
stem(t, x, 'r', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Unit Impulse Discrete')

% Unit Step
for i = 1:t_leng

    if t(i) < 0
```

```
        y(i) = 0;
    else
        y(i) = 1;
    end

end

subplot(3, 2, 3)
plot(t, y, 'b', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Unit Step Continuous')
subplot(3, 2, 4)
stem(t, y, 'b', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Unit Step Discrete')

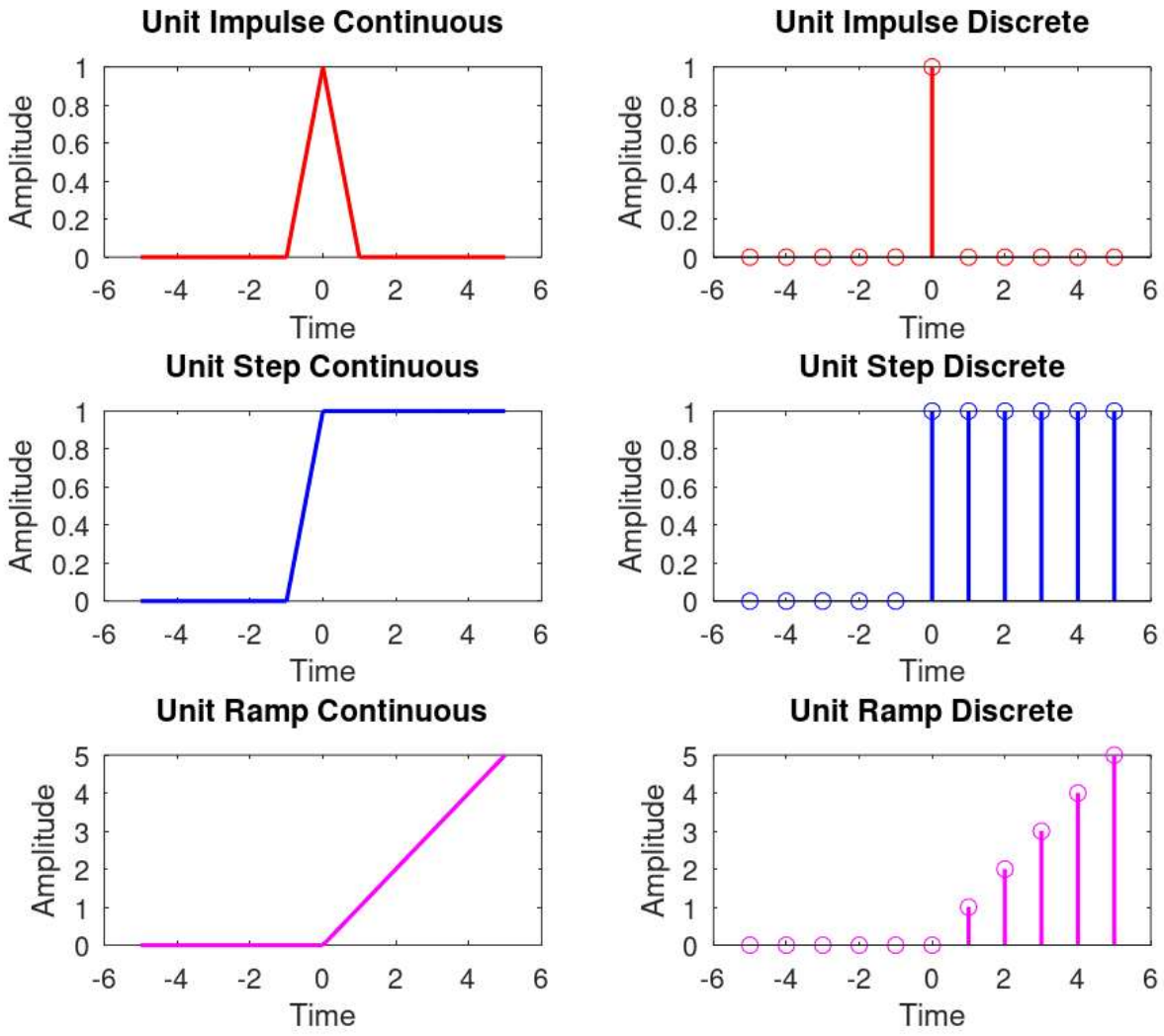
% Unit Ramp
for i = 1:t_leng

    if t(i) >= 0
        z(i) = t(i);
    else
        z(i) = 0;
    end

end

subplot(3, 2, 5)
plot(t, z, 'm', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Unit Ramp Continuous')
subplot(3, 2, 6)
stem(t, z, 'm', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Unit Ramp Discrete')
```

## Output:


Unit Impulse Continuous
Amplitude
Time
Unit Impulse Discrete
Amplitude
Time
Unit Step Continuous
Amplitude
Time
Unit Step Discrete
Amplitude
Time
Unit Ramp Continuous
Amplitude
Time
Unit Ramp Discrete
Amplitude
Time

## b. Generation of exponential and sinusoidal signal using MATLAB.

## Code:

```
clc;
clear all;
close all;

t = [-5:1:5];

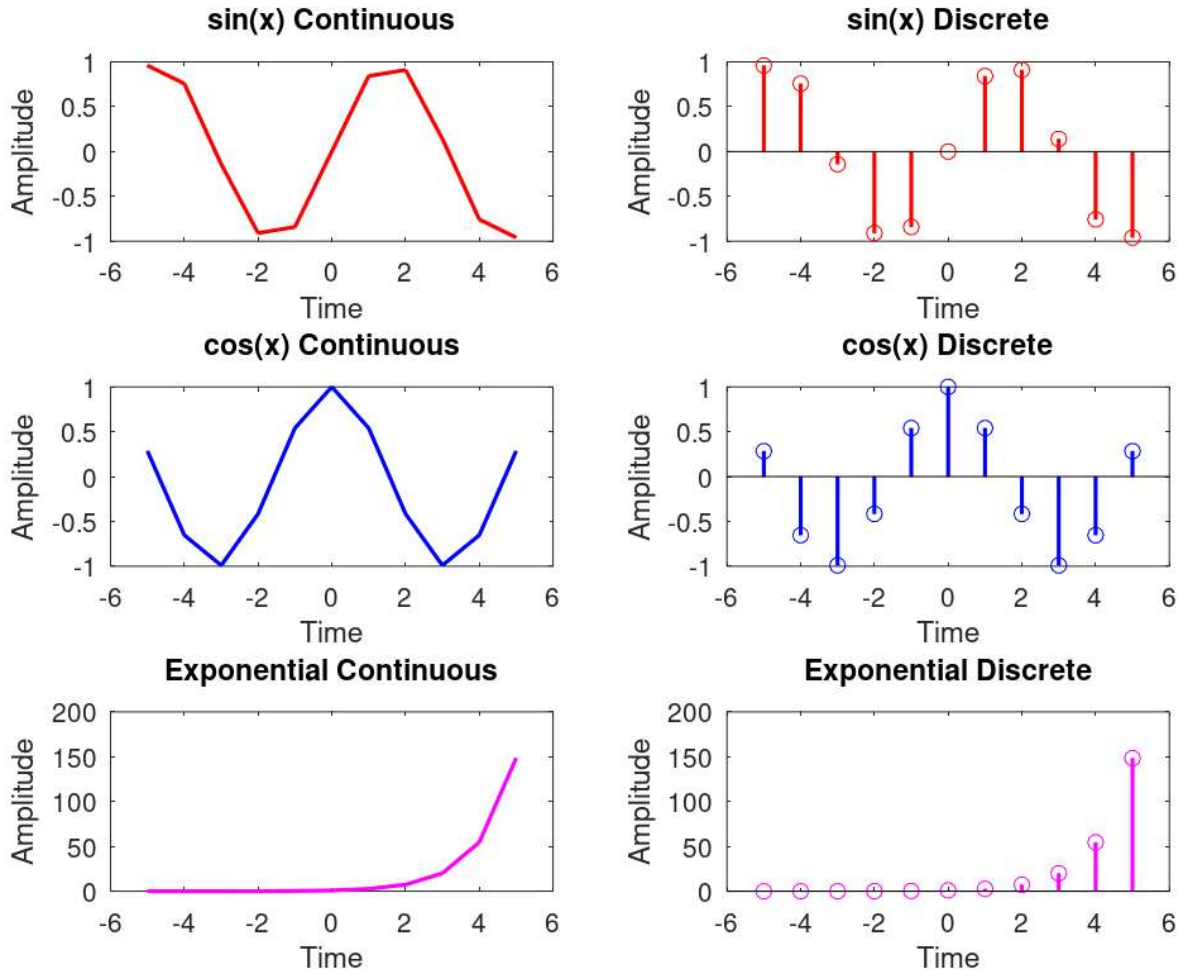
%Sine Function
x = sin(t);
subplot(3, 2, 1)
plot(t, x, 'r', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('sin(x) Continuous')
subplot(3, 2, 2)
stem(t, x, 'r', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('sin(x) Discrete')

%Cosine Function
y = cos(t);
subplot(3, 2, 3)
plot(t, y, 'b', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('cos(x) Continuous')
subplot(3, 2, 4)
stem(t, y, 'b', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('cos(x) Discrete')

%Exponential Function
z = exp(t);
subplot(3, 2, 5)
plot(t, z, 'm', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
```

```
title('Exponential Continuous')
subplot(3, 2, 6)
stem(t, z, 'm', 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Exponential Discrete')
```

## Output:



# Experiment : 3

## AIM:- Plot the linear convolution of two sequences.

## Code:

```
clc;
clear all;
close all;

x = [1, 2, 3];
h = [1, 2];

N1 = length(x);
N2 = length(h);
N = N1 + N2 - 1;

y = zeros(1, N);

for i = 1:N
    for j = 1:N1
        if (i - j + 1 >= 1) && (i - j + 1 <= N2)
            y(i) = y(i) + x(j) * h(i - j + 1);
        end
    end
end

disp('Convolution result using for loops:');
disp(y);
disp('Convolution result using built in function:');
disp(conv(x, h));

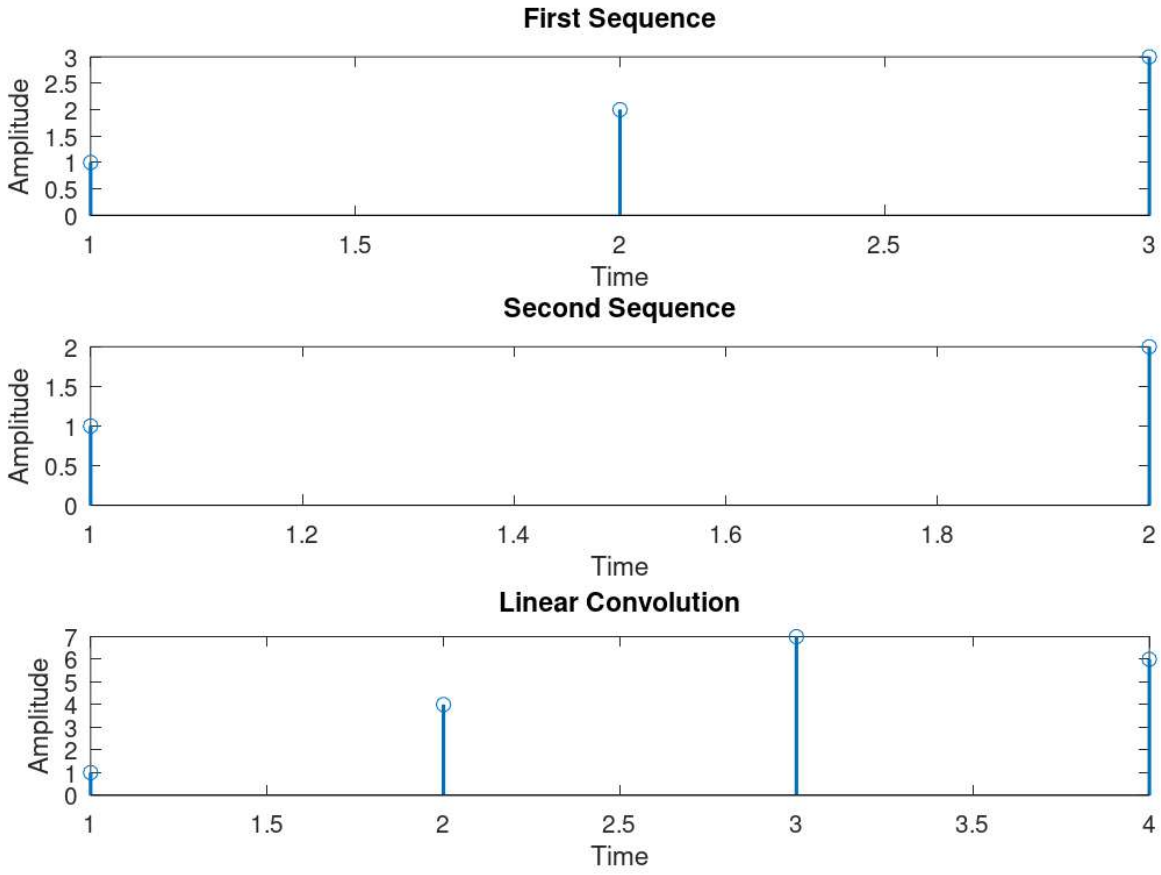
subplot(3, 1, 1)
stem(x, 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('First Sequence')

subplot(3, 1, 2)
stem(h, 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Second Sequence')
```

```
subplot(3, 1, 3)
stem(y, 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Linear Convolution')
```

## Output:

```
Convolution result using for loops:
   1   4   7   6
Convolution result using built in function:
   1   4   7   6
```



# Experiment : 4

## AIM:- Plot the linear convolution of two sequences.

## Code:

```
clc;
clear all;
close all;

x = [1, 2, 3];
h = [1, 2];
y = xcorr(x, h);

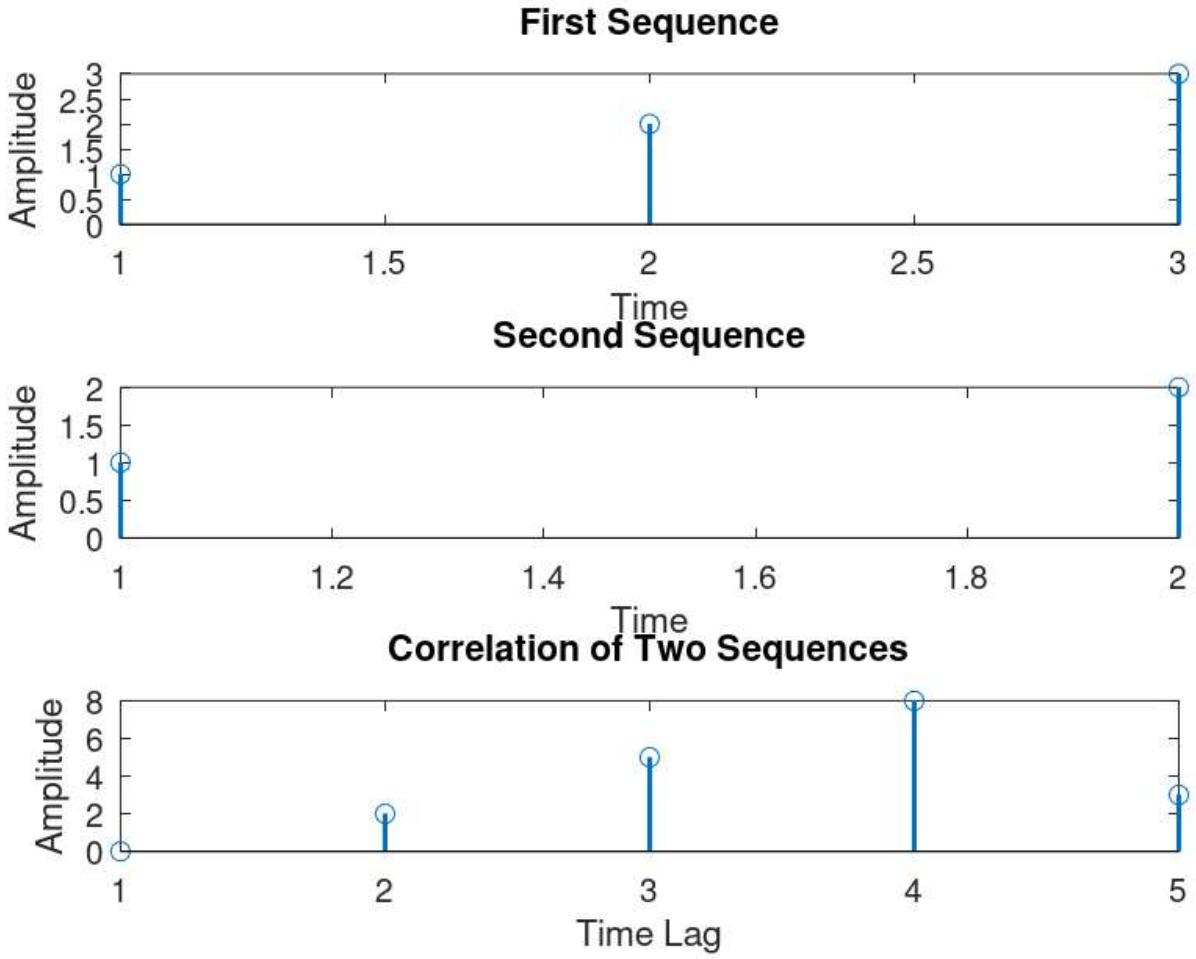
disp('Correlation result using built in function:');
disp(y);

subplot(3, 1, 1)
stem(x, 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('First Sequence')

subplot(3, 1, 2)
stem(h, 'LineWidth', 1.5);
xlabel('Time')
ylabel('Amplitude')
title('Second Sequence')

subplot(3, 1, 3)
stem(y, 'LineWidth', 1.5);
xlabel('Time Lag')
ylabel('Amplitude')
title('Correlation of Two Sequences')
```

**Output:**


First Sequence
Amplitude
Time
Second Sequence
Amplitude
Time
Correlation of Two Sequences
Amplitude
Time Lag

# Experiment : 5

## AIM:- Plot the magnitude and phase spectra of a signal using Fourier transforms.

## Code:

```
clc;
clear all;
close all;

syms t;

f = 3 * exp(-3*t) * heaviside(t);
F = fourier(f);

w1 = -2*pi : pi/50 : 2*pi;
w2 = -2*pi : pi/100 : 2*pi;

F1 = subs(F,w1);
F2 = subs(F,w2);

F1_mag = abs(F1);
F1_phase = angle(F1);

F2_mag = abs(F2);
F2_phase = angle(F2);

% MAGNITUDE
subplot(2,2,1);
plot(w1,F1_mag,'r','LineWidth',1);
grid on;
box on;
xlabel('\omega');
ylabel('|F(\omega)|');
title('Magnitude Spectrum - \omega step = \pi/50');
legend('\pi/50');

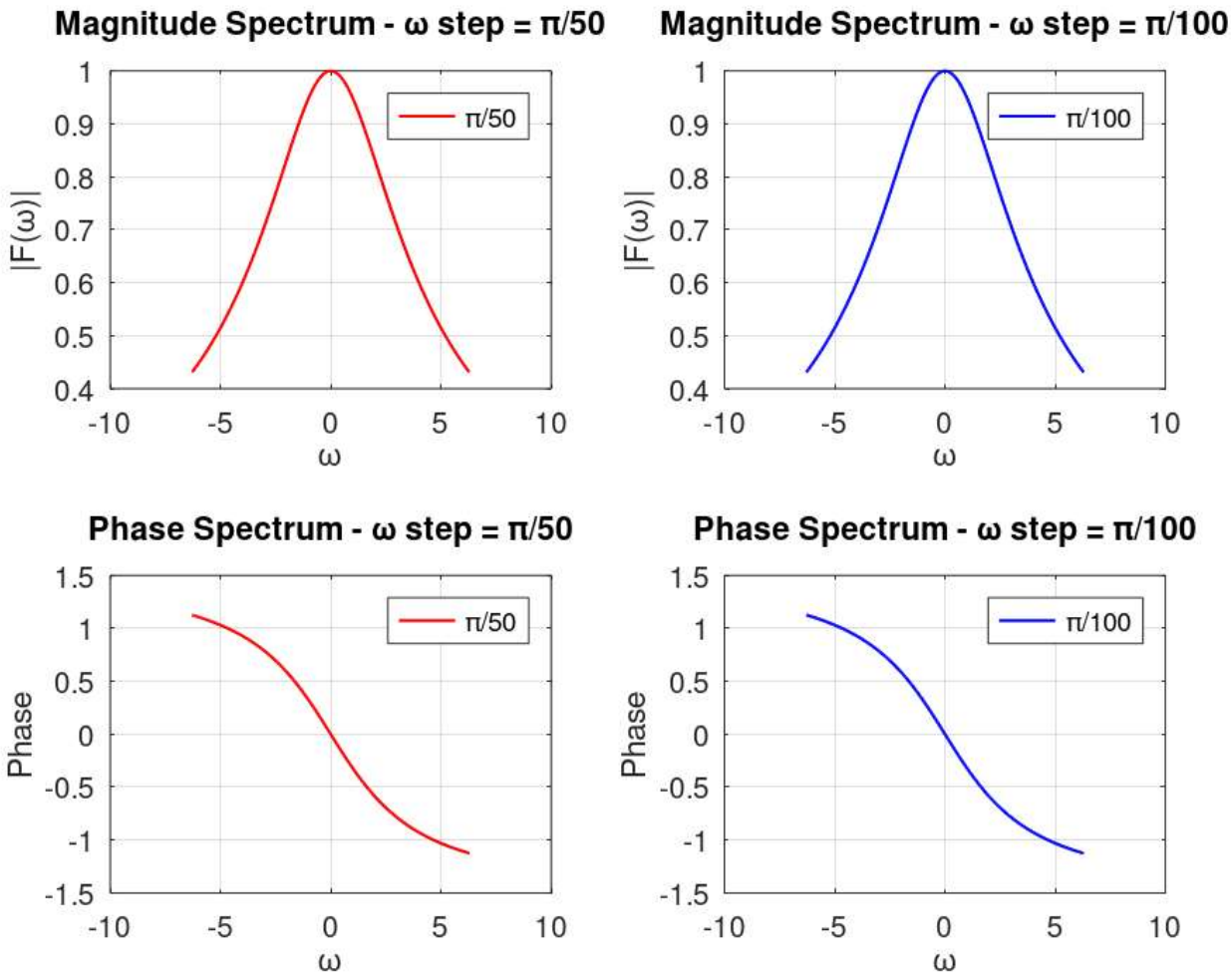
subplot(2,2,2);
plot(w2,F2_mag,'b','LineWidth',1);
grid on;
box on;
```

```
xlabel('\omega');
ylabel('|F(\omega)|');
title('Magnitude Spectrum - \omega step = \pi/100');
legend('\pi/100');

% PHASE
ubplot(2,2,3);
plot(w1,F1_phase,'r','LineWidth',1);
grid on;
box on;
xlabel('\omega');
ylabel('Phase');
title('Phase Spectrum - \omega step = \pi/50');
legend('\pi/50');

subplot(2,2,4);
plot(w2,F2_phase,'b','LineWidth',1);
grid on;
box on;
xlabel('\omega');
ylabel('Phase');
title('Phase Spectrum - \omega step = \pi/100');
legend('\pi/100');
```

## Output:


Magnitude Spectrum - ω step = π/50
|F(ω)|
ω
π/50
Magnitude Spectrum - ω step = π/100
|F(ω)|
ω
π/100
Phase Spectrum - ω step = π/50
Phase
ω
π/50
Phase Spectrum - ω step = π/100
Phase
ω
π/100